

Preserved-Identity

Rearranging an Image’s Own Pixels to Take the Form of Another

Yash Yadav

June 2025

Abstract

Given a *source* image (a “bag of pixels”) and a *target* image (a form to reproduce), this project produces an output that **looks like the target but is built entirely from the source’s own pixels**. The output is a strict *permutation* of the source — every pixel used exactly once — so it preserves the source’s exact colour histogram, contrast, and brightness while taking on the target’s spatial structure. We formulate this as an optimal-transport assignment problem, solve it at scale with a recursive median-split approximation, and apply it in a source-anchored portrait pipeline that morphs a photo’s subject onto a target face while keeping the original image intact.

1 Objective

Given a **source** image (e.g. a photo of a person outdoors) and a **target** image (e.g. a portrait), produce an output that resembles the target but is composed **entirely from the source’s own pixels**.

The defining constraint is that the output is a **permutation** of the source: every source pixel is used exactly once, none invented and none dropped. Consequently the output keeps the source’s exact **colour histogram, contrast, and brightness** — it is literally the same pixels, relocated in space. Dark source pixels migrate to where the target is dark (hair, shadows), bright pixels to highlights, and so on.

This is **not** a photomosaic (which tiles a picture out of many small *separate* images). It is a single-image **pixel-rearrangement / optimal-transport** problem.

2 Problem Formulation

Let the source contribute N pixels and the target define N positions. We seek a bijection σ (a one-to-one assignment) from source pixels to target positions that minimises the total perceptual colour distance:

$$\min_{\sigma \in \mathcal{S}_N} \sum_{p=1}^N \text{dist}(c_{\sigma(p)}^{\text{src}}, c_p^{\text{tgt}}), \quad (1)$$

where $\mathcal{S}_N$ is the set of all permutations of N elements, c^{src} and c^{tgt} are source and target colours, and $\text{dist}(\cdot, \cdot)$ is Euclidean distance in **CIELAB** colour space (perceptually uniform, closer to how the eye judges colour than raw RGB). Solving this exactly is the **linear assignment problem**.

3 The Matching Engine (src/pixel_rearrange.py)

Three interchangeable solvers trade quality for scale:

Method	Technique	Quality	Scale
exact	Hungarian algorithm	Optimal	$\leq 64 \times 64$ ($O(N^3)$)
recursive	Median-split optimal transport	$\sim$ few % from optimal	Megapixels (1024^2)
sorted	Lightness sort	Coarse	Instant

Recursive median-split OT is the workhorse that made the project scale. It recursively partitions both pixel clouds along their highest-variance colour axis, splitting each at the median so the halves keep equal counts, and solves the small blocks ($\leq \texttt{leaf}$ pixels) exactly with the Hungarian algorithm. This preserves a true global bijection while running in $\sim O(N \log N)$ plus cheap leaf solves — turning an intractable $O(N^3)$ assignment into one that handles full photographs.

Engine extras.

- **Recolour knob** (`--recolor`): optionally nudge rearranged pixels toward the true target colours. `luma` mode adopts the target’s lightness/contrast while keeping the source’s chroma; `blend` is a plain RGB blend. `0` = pure shuffle (histogram perfectly preserved).
- **Priority mode**: the most important target positions (face/edges) claim the globally best-fitting source pixels first, while remaining a strict permutation.
- **Aspect handling**: the output follows the target’s native aspect ratio.

4 Source-Anchored Portrait Pipeline (`src/portrait.py`)

The headline application keeps the source image intact (size, palette, contrast) and morphs **only its subject** onto the target’s form, on a clean background — a **budget-driven, gap-free** rearrangement:

1. **Partition the source by count.** Segment into the dominant background (e.g. sky) and everything else (the “object pool”), giving N_{obj} object pixels and N_{bg} background pixels.
2. **Size the subject to the budget.** Take the target’s subject silhouette (background removed, auto-oriented upright) and **scale it so its area equals** N_{obj} , centred in the frame. The remaining border then equals N_{bg} exactly — counts match on both sides.
3. **Overshoot + erode.** The subject is scaled to slightly over-fill, then eroded to the exact budget. Growing a region (dilation) produces blobby halos, whereas trimming an oversized subject (erosion) stays clean.
4. **Render the subject.** The N_{obj} object pixels are rearranged onto the subject guide with **tonal cluster sub-matching** (dark $\rightarrow$ hair/shadow, mid $\rightarrow$ skin) for a face-like read.
5. **Smooth background.** The N_{bg} background pixels fill every border position, each taking its **nearest original background colour** — reproducing the sky’s gradient and blending the vacated centre seamlessly.

Result: every source pixel used once, every output position filled — no empty/jagged gaps, no halos, identical histogram/contrast/brightness to the source, subject cleanly centred.

5 Segmentation (src/segment.py)

Semantic, **not** colour-based — which lets a blue book held against a blue wall stay foreground while the wall becomes background:

- **Mask R-CNN** (COCO instances) — unions every confidently detected object. A `person_only` flag restricts to the `person` class for the *target* subject guide, so nearby furniture (a chair behind the head) is never merged into the silhouette. The source still unions all objects to protect held items.
- **DeepLabV3** (person class) — fallback when Mask R-CNN is unavailable.
- **GrabCut** (OpenCV) — final offline fallback.
- **Face-aware auto-orientation** (MTCNN) — detects the most confident face across the four right-angle rotations and rotates the target upright before fitting.
- **Largest-connected-component** filtering removes stray detached mask blobs.

6 Module Map

File	Role
src/pixel_rearrange.py	Optimal-transport assignment engine (exact / recursive / sorted), recolour, priority.
src/portrait.py	Source-anchored portrait pipeline (the main product).
src/segment.py	Mask R-CNN / DeepLabV3 / GrabCut masks, face detection & auto-orient.
src/saliency.py	Target importance map (centre bias + edge energy + face box) for <code>--priority</code> .
src/cli.py	CLI for plain whole-image rearrangement.
scripts/demo_rearrange.py	Synthetic tree → face proof-of-concept.

7 How to Run

Source-anchored portrait (main):

```
python -m src.portrait \  
  --source data/source.jpeg \  
  --target data/target.jpg \  
  --out results/portrait.png \  
  --max-dim 1600 --seg maskrcnn
```

Plain whole-image rearrangement:

```
python -m src.cli \  
  --source data/source.jpg --target data/target.jpg \  
  --out results/out.png --size 1024 --space lab --recolor 0.3
```

Install: `pip install -r requirements.txt` (numpy, scipy, Pillow, scikit-image, plus opencv-python / facenet-pytorch / torch / torchvision for the portrait pipeline).

8 Choosing a Good Source (Matters Most)

Because the output is a *pure rearrangement*, its colours are **locked to the source’s palette** — no algorithm can create colours the source lacks. A mostly-white source can never produce dark hair; a dark source can never make a bright sky. For best results pick a source whose brightness/colour distribution **overlaps the target’s** (mid-tones for skin, some dark pixels for hair/shadows, etc.).

9 Development Journey

1. **Course correction.** The repository began as an identity-aware *photomosaic* (face tiles + embeddings + LPIPS). That solved the wrong problem; it was scrapped in favour of single-image pixel rearrangement.
2. **Proof of concept.** Exact Hungarian assignment in RGB on small synthetic images (tree → face) confirmed the histogram-preserving rearrangement worked.
3. **Quality.** Switched matching to perceptual CIELAB — visibly smoother, more faithful.
4. **Scale.** Added the recursive median-split OT solver to reach megapixel images.
5. **Portrait pipeline.** Built segmentation plus the source-anchored portrait so the subject takes the target’s form while the source is preserved.
6. **Final polish.** Replaced the frozen-background approach with the budget-driven partition (gap-free, smooth background, subject centred); fixed a blobby halo via overshoot-then-erode and largest-component filtering; and fixed background leakage into the subject by restricting the target mask to `person_only`.

10 Limitations & Future Work

- **Palette lock** is fundamental — resemblance depends on source/target palette overlap.
- The recursive solver is an *approximation* of optimal transport (a few % from optimal); Sinkhorn could close the gap if needed.
- Segmentation quality bounds portrait quality; unusual poses or occlusions can still mis-segment. Soft-alpha matting would refine subject boundaries.
- Background smoothing assumes a single dominant background region (works well for sky/wall; busy backgrounds are less clean).

11 Status

Complete. The pipeline produces clean, gap-free source-anchored portraits: the target’s form rendered entirely from the source’s own pixels, subject centred on a smooth background, with the source’s exact histogram, contrast, and brightness preserved.